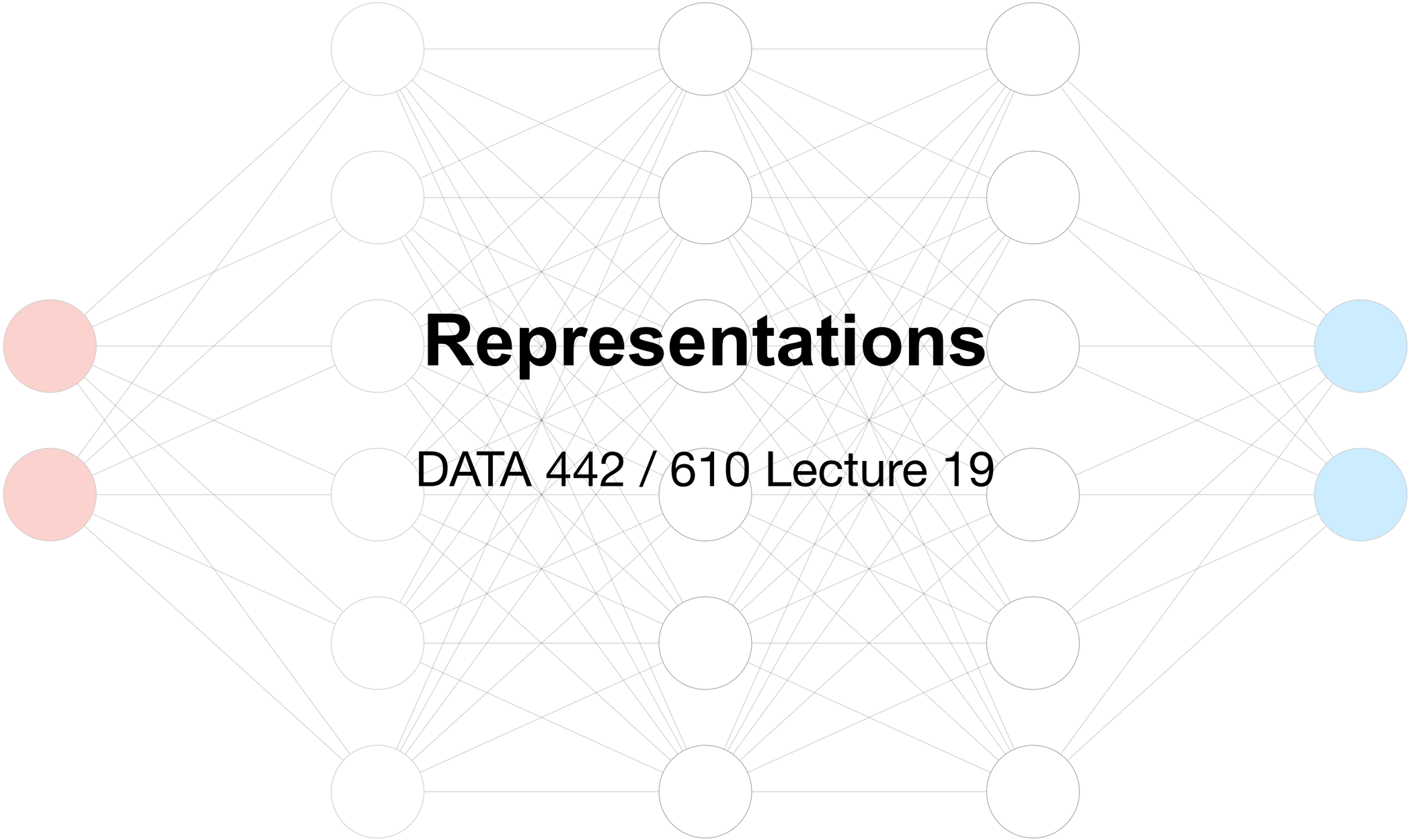


Representations

DATA 442 / 610 Lecture 19

Transformers

Attention is all you need (2017)

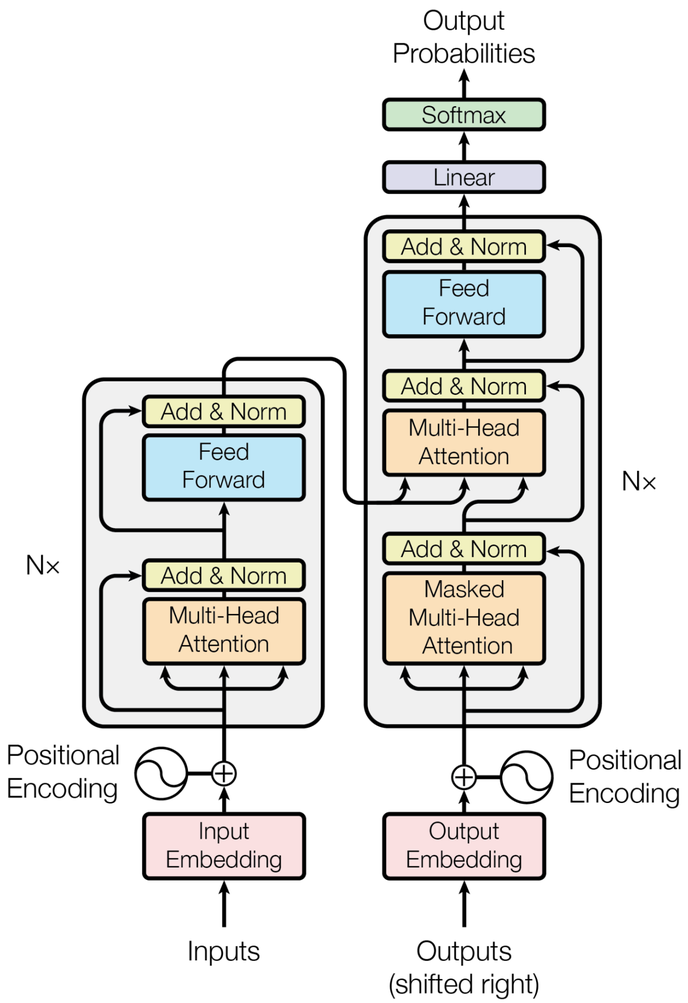


Figure 1: The Transformer - model architecture.

An image is worth 16x16 words (2021)

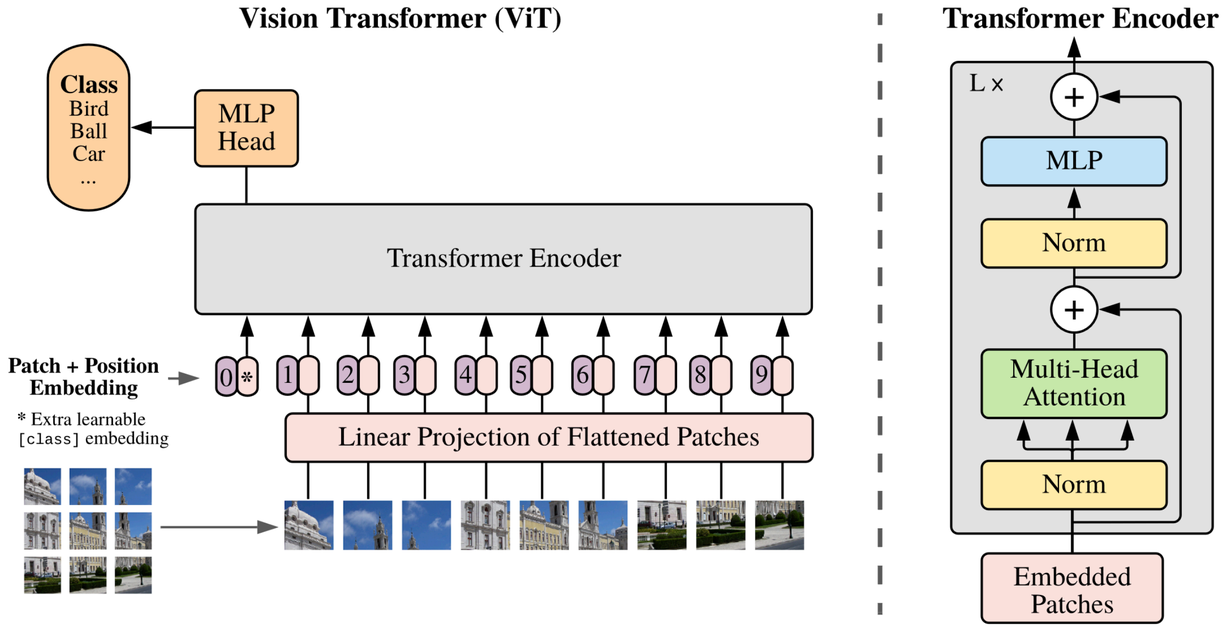


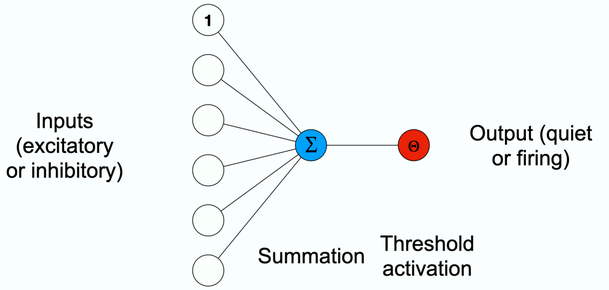
Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

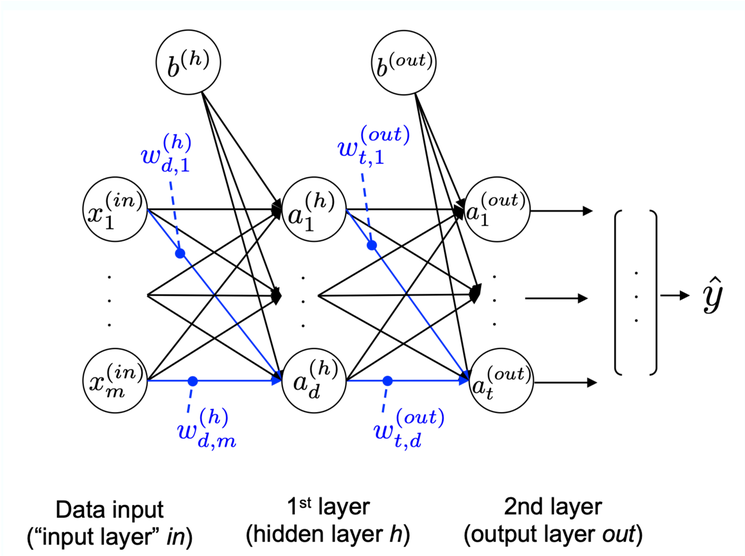
ArXiv:1706.03762
ArXiv:2010.11929

Course outline

September: Fundamentals

Artificial Neuron (McCulloch/Pitts, 1943)



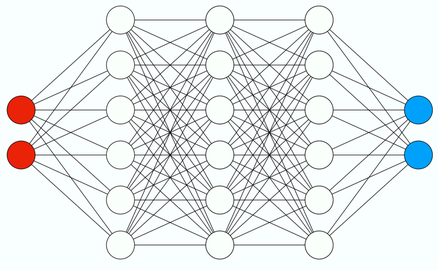


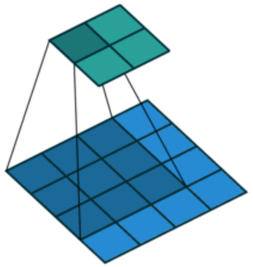
Data input
(“input layer” *in*)

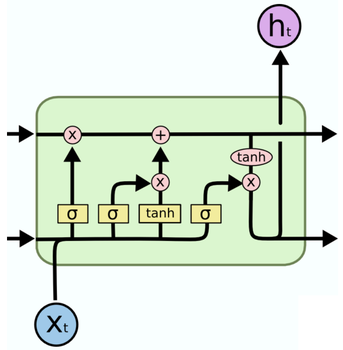
1st layer
(hidden layer *h*)

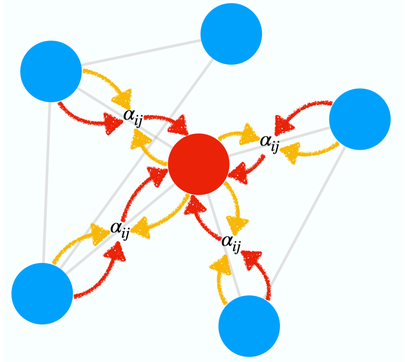
2nd layer
(output layer *out*)

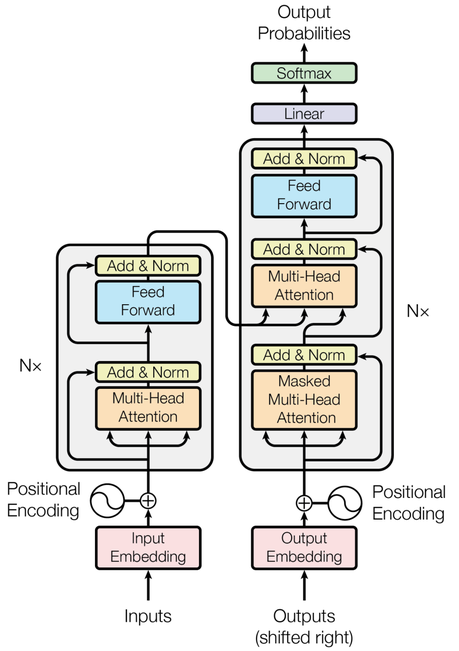
October: Architectures



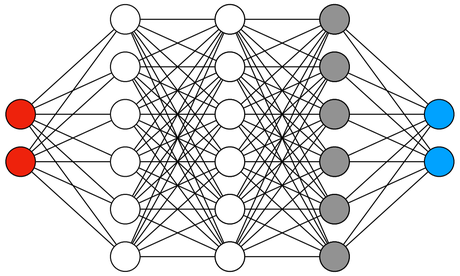


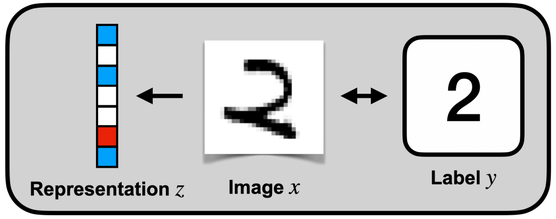


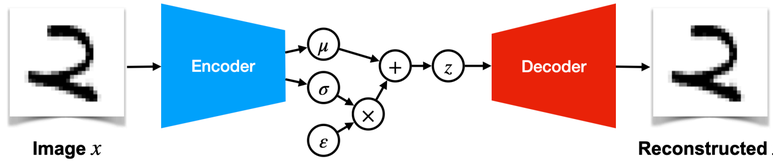


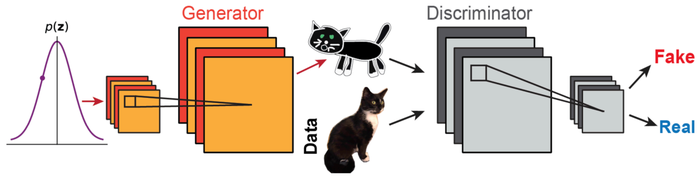


November: Frameworks









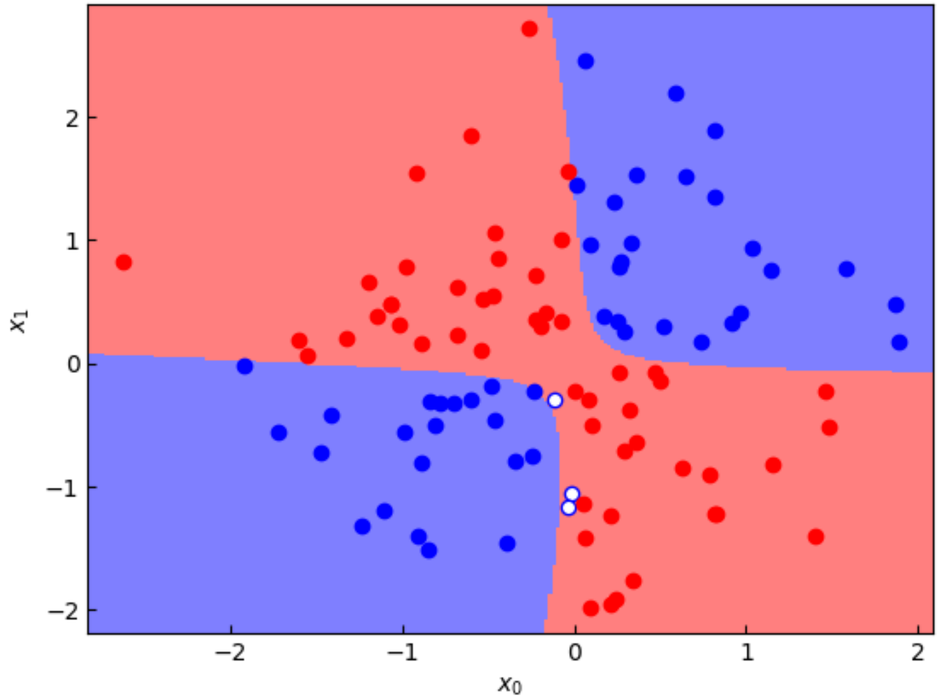
Outline

- Review of feature LR and the kernel trick
- What are representations?
- How do we learn representations?

Relevant Reading: *Deep Learning* Chapter 15, Alex Alemi's notes

Nonlinear optimization

- Optimizing nonlinear functions is challenging because optimization methods for linear functions may no longer apply
- One way around this is to do *linear* optimization on a nonlinear transformation of the inputs $z = \phi(x)$ (polynomial LR)
- Another way is to exploit the **kernel trick**



$$f(x) = b + \sum_i \alpha_i \, \phi(x)^\top \phi(x^i)$$

$$\downarrow$$

$$f(x) = b + \sum_i \alpha_i \, k_\phi(x, x^i)$$

Nonlinear optimization

Without knowing the “right” feature function $\phi(x)$, the **kernel trick** lets us *implicitly* compute dot products between high-(or infinite-)dimensional feature vectors

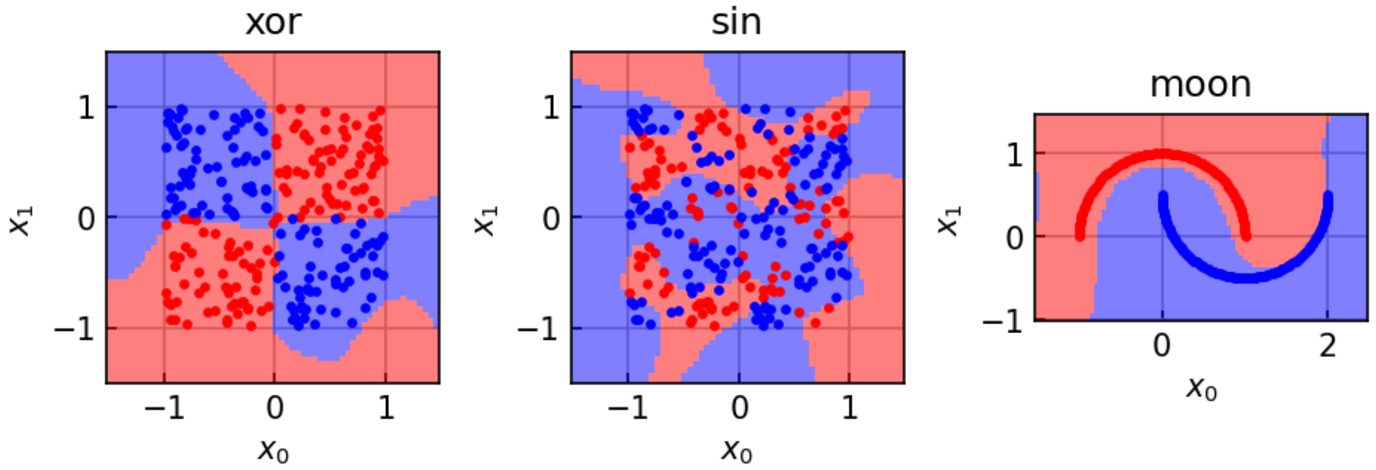
Drawback #1: Training requires inverting an $N \times N$ matrix [$O(N^3)$]

$$\vec{\alpha} = (K + \lambda I)^{-1} \hat{Y}$$

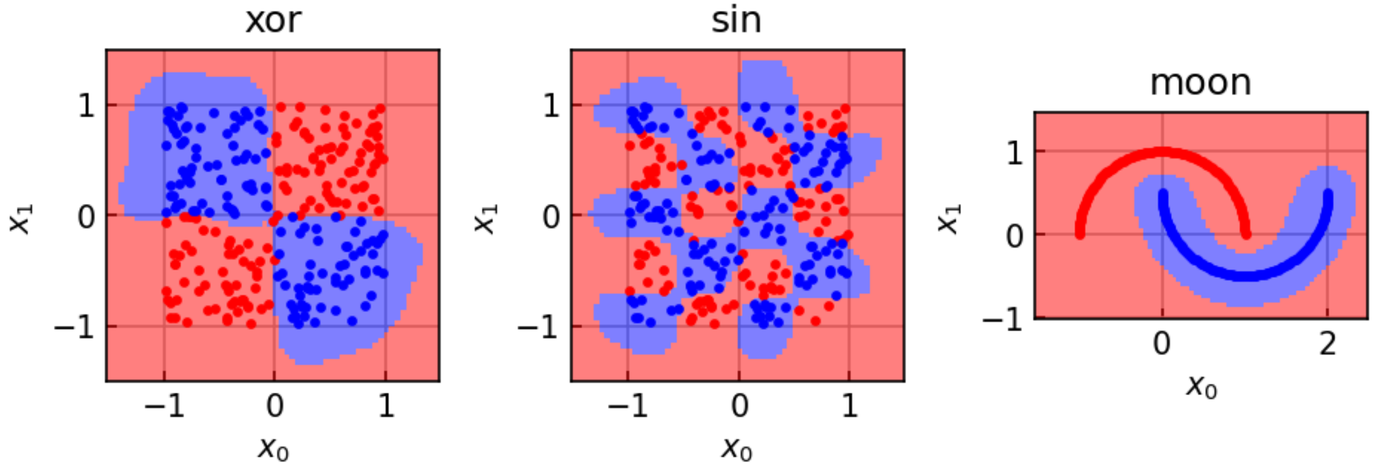
Drawback #2: Inference requires N kernel function evaluations

$$y(x) = \sum_{i=1}^{N} \alpha_i \, k(x_i, x)$$

Degree-10 Polynomial LR



Kernel Ridge Regression



Nonlinear optimization

Degree-10 Polynomial LR

But do I always need an ***infinite-dimensional*** feature vector $\phi(x)$?

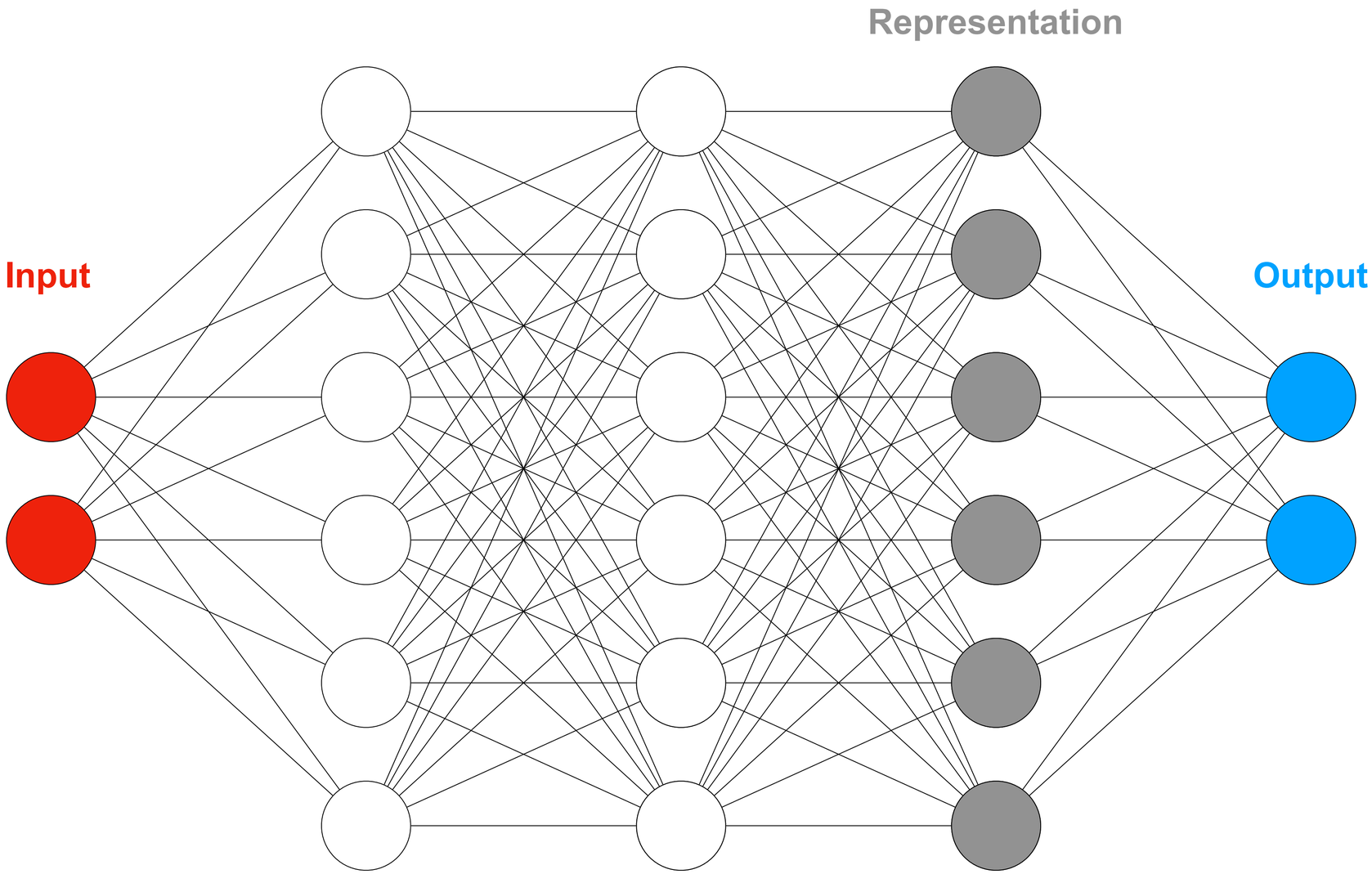


$$\vec{\alpha} = (K + \lambda I)^{-1}\hat{Y}$$

Kernel Ridge Regression

Deep neural networks learn nonlinear features $\phi(x)$ from data

What is a representation?



A **representation** contains **features** that describe the data

Why care about representations?

Labeled data? Scarce, $$$$$$$$$

Unlabeled data? Abundant, $ (or free)

Unsupervised pretraining aims to find useful patterns or representations without a chosen prediction task

Semi-supervised learning uses large unlabeled datasets to learn representations useful for prediction on smaller labeled datasets

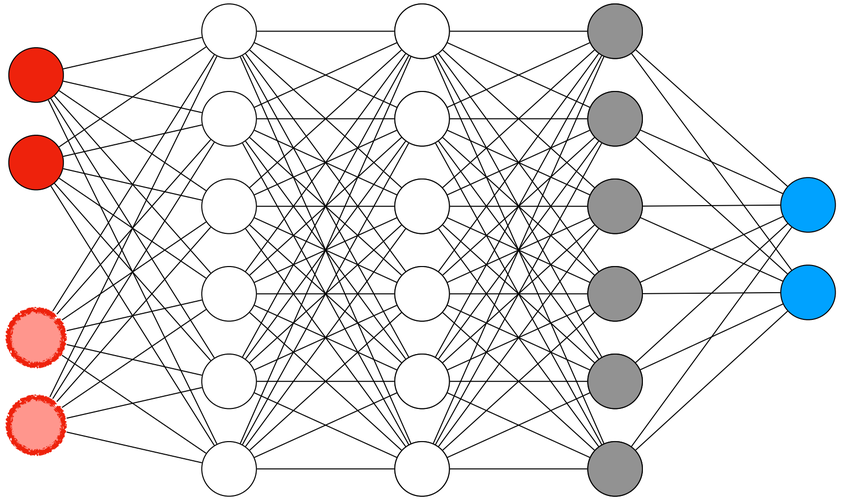
Self-supervised learning creates a *pretext task* on large unlabeled data to learn representations, prior to supservised fine-tuning

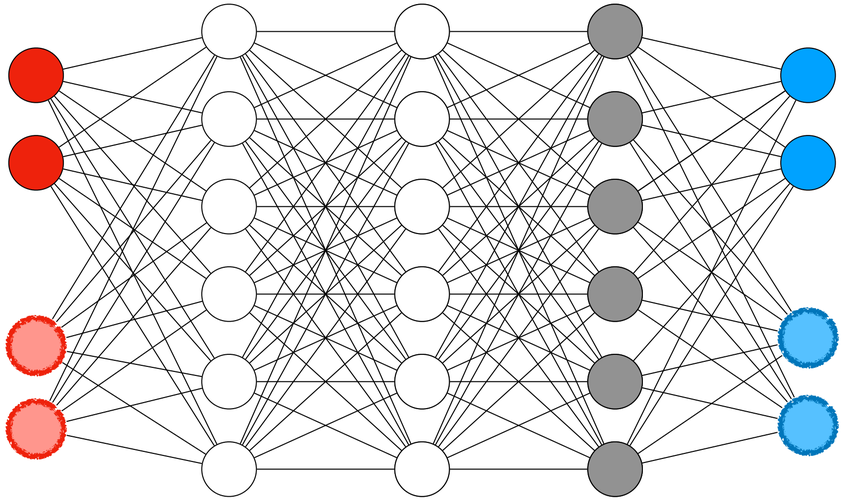
Transfer Learning vs. Domain Adaptation

Domain Adaptation: model learns representations that are consistent across input domains

Transfer Learning: model learns representations that can adapt to multiple tasks

Few/One/Zero Shot Learning: model learns using minimal number of labeled examples





What is a “good” representation?

A *good* representation…

…excels at a range of downstream prediction tasks

…is easy to model and maybe interpret

…captures causes behind data variation but not underlying noise

…uses a minimal number of independent features

Let’s go examine some representations



What is a “good” representation?

A *good* representation…

…excels at a range of downstream prediction tasks

…is easy to model and maybe interpret

…captures causes behind data variation but not underlying noise

…uses a minimal number of independent features

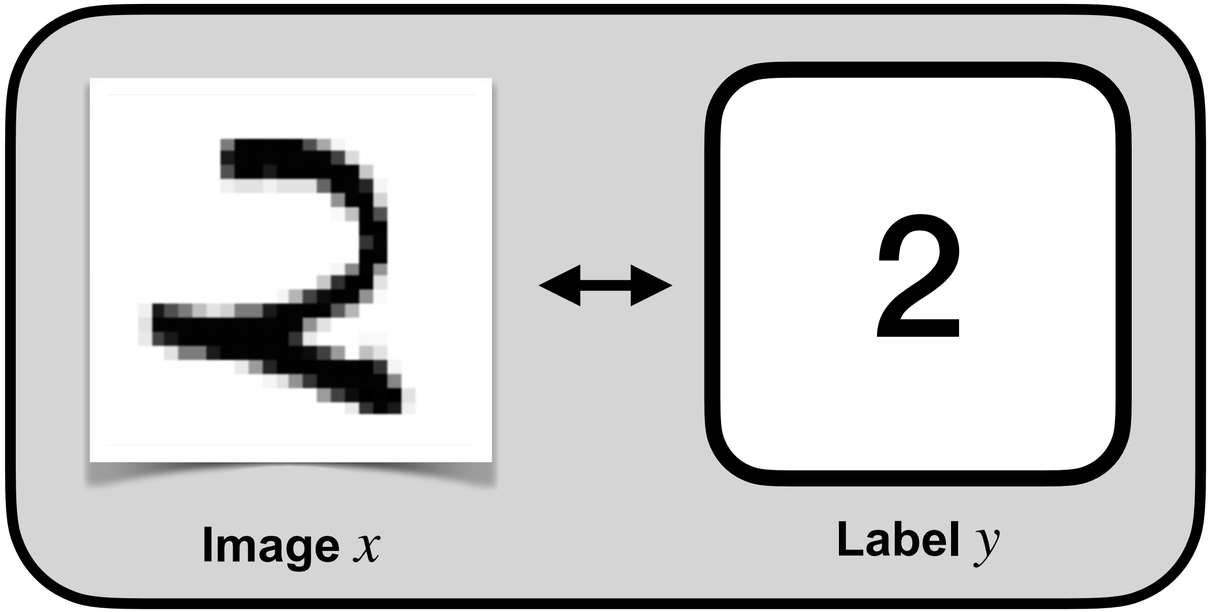
How do we learn *good* representations?

How do we learn?

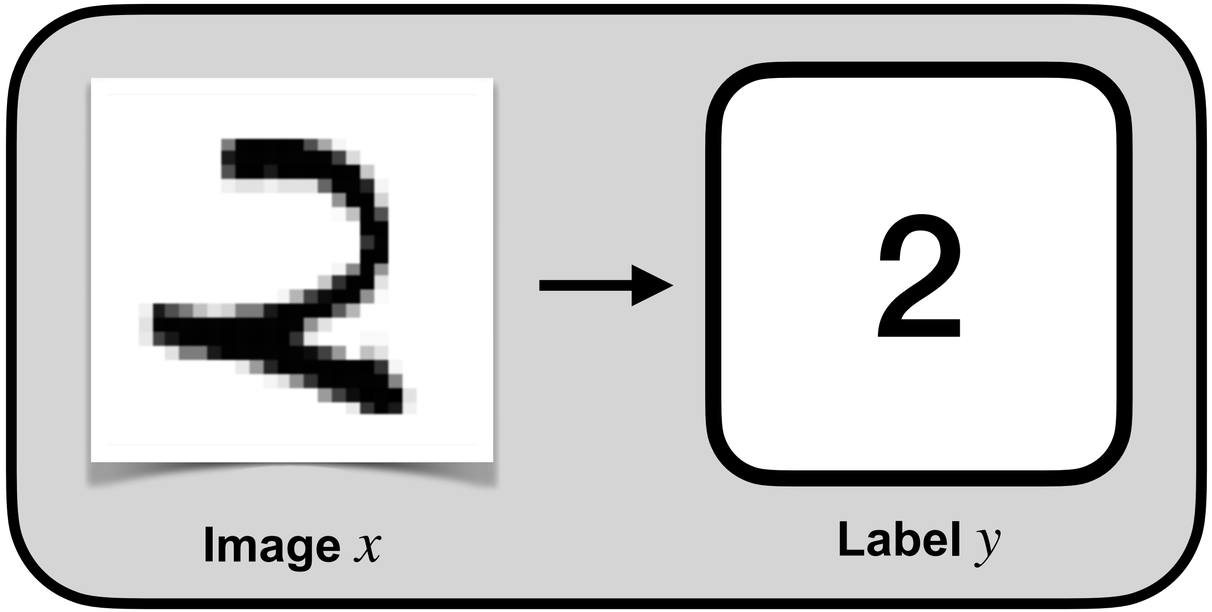
1. Draw a diagram of the data we have P
2. Draw a diagram of how we would like to model the data Q

Example: Supervised Classification

Real world $P : p(x, y)$



Fantasy world $Q : q_\theta(x, y) = p(x)\, q_\theta(y \mid x)$



How do we learn?

1. Draw a diagram of the data we have P
2. Draw a diagram of how we would like to model the data Q
3. Minimize the **Kullback-Leibler Divergence** $D_{KL}(P \parallel Q)$????

Kullback-Leibler Divergence

Definition: $D_{KL}\left[P \,\|\, Q\right] \equiv \mathbb{E}_{x\sim p(x)}\left[\log \frac{p(x)}{q(x)}\right] \geq 0$

What is it?

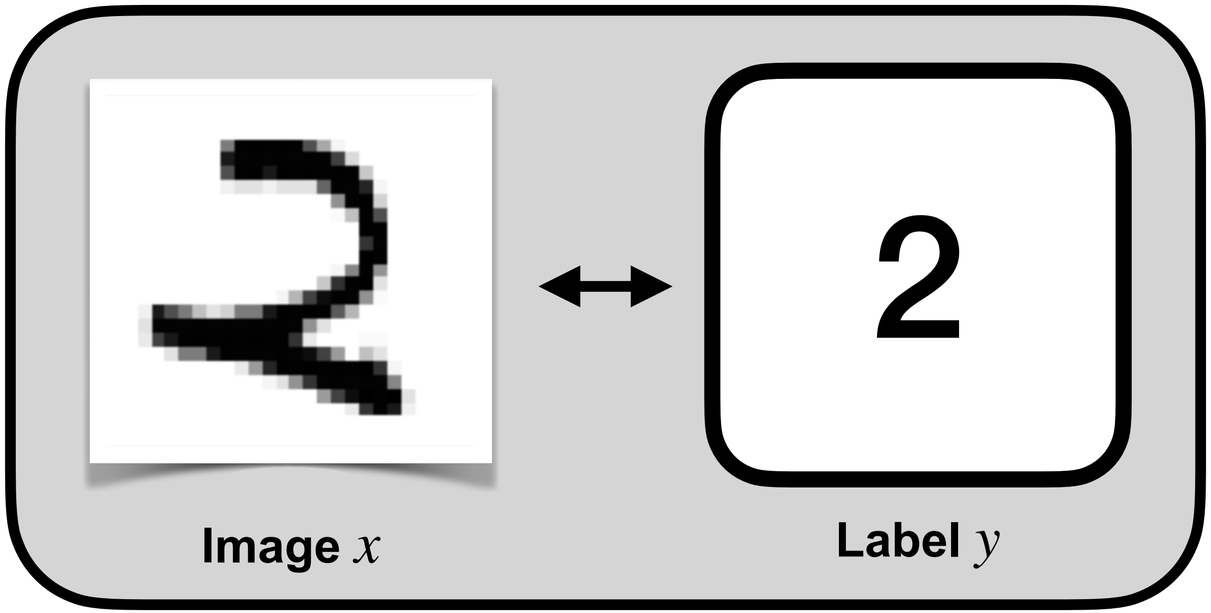
A measure of distance between two distributions p, q

A measure of "surprise" when observing samples drawn from p if expecting a different distribution q

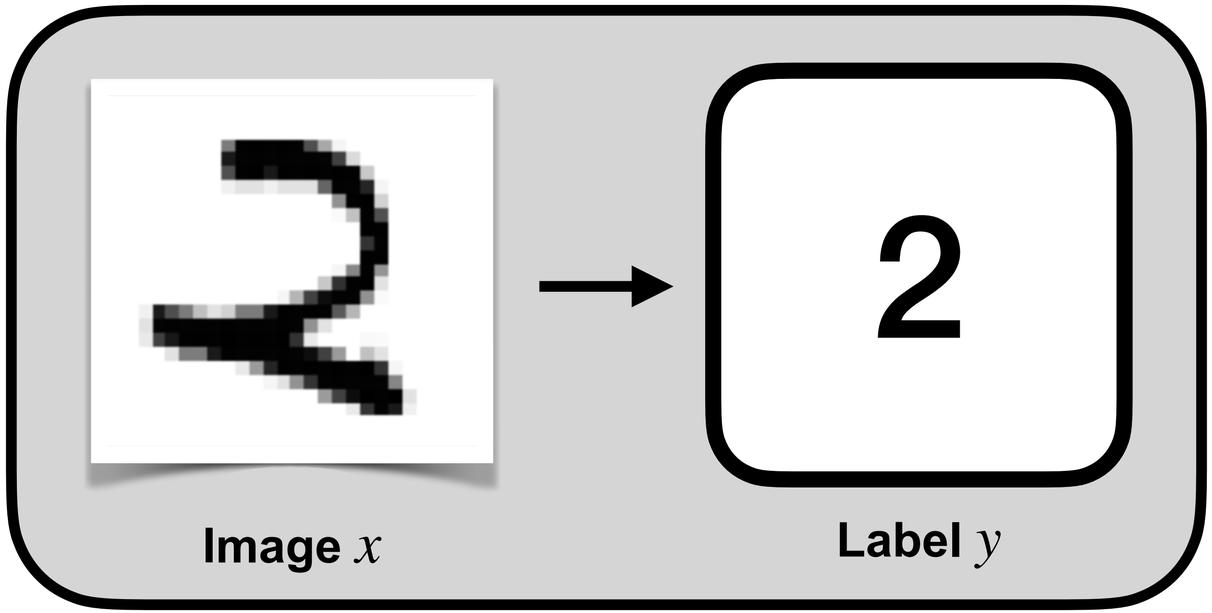
A measure of our ability to discern the "real world" p from a proposed model q

Example: Supervised Classification

Real world $P : p(x, y)$



Fantasy world $Q : q_\theta(x, y) = p(x)\, q_\theta(y \mid x)$



Minimize KL Divergence

$$D_{KL}\big(P \,\|\, Q\big) = D_{KL}\big(p(x, y) \,\|\, p(x)\, q_\theta(y \mid x)\big)$$

$$= \mathbb{E}_{x,y\sim p(x,y)}\left[\log \frac{p(x, y)}{p(x)\, q_\theta(y \mid x)}\right] = \mathbb{E}_p\left[\log \frac{p(y \mid x)}{q_\theta(y \mid x)}\right]$$

Minimize negative log likelihood

$$= -\,\mathbb{E}_p\big[\log q_\theta(y \mid x)\big] + \mathbb{E}_p\big[\cancel{\log p(y \mid x)}\big]$$

Example: Supervised Classificatio

Real world $P : p(x, y)$



2

Image x

Label y

Fantasy world

2

x

But where are my representations???

Minimize K

$$\left(p(x, y) \,\|\, p(x)\, q_\theta(y \,|\, x)\right)$$

ood

$$= \mathbb{E}_{x,y \sim p(x,y)} \left[\log \frac{p(x, y)}{p(x)\, q_\theta(y \,|\, x)} \right] = \mathbb{E}_p \left[\log \frac{p(y \,|\, x)}{q_\theta(y \,|\, x)} \right]$$

$$= - \mathbb{E}_p \left[\log q_\theta(y \,|\, x) \right] + \mathbb{E}_p \left[\cancel{\log p(y \,|\, x)} \right]$$

How do we learn representations?

1. Draw a diagram of the data we have P
2. Augment the diagram with representations
3. Draw a diagram of how we would like to model the data and representations Q
4. Minimize the **Kullback-Leibler Divergence** $D_{KL}(P \parallel Q)$
5. Adjust seasonings, salt to taste

Example: Supervised learning with representations

$$D_{KL}\big(P \,\|\, Q\big) = D_{KL}\big(p(x,y)\; p_\theta(z\,|\,x) \;\|\, p(x)\, q(z)\, q_\phi(y\,|\,z)\big)$$

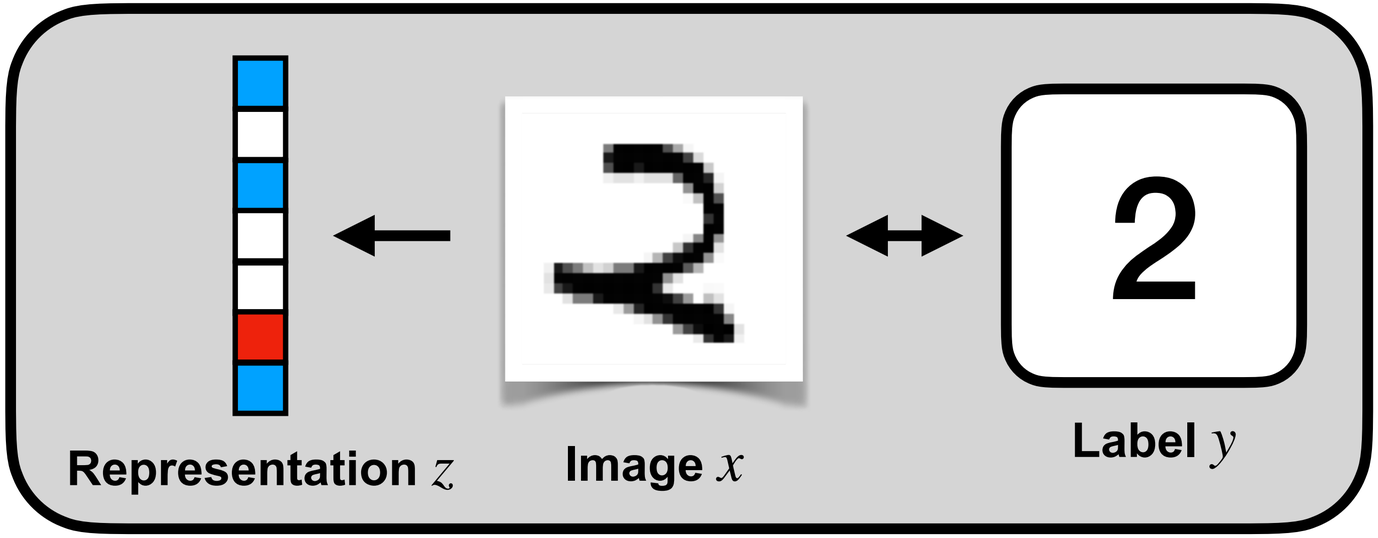
$$= \mathbb{E}_{x,y\sim p; z\sim p_\theta}\left[\log \frac{p(x,y)\, p_\theta(z\,|\,x)}{p(x)\, q(z)\, q_\phi(y\,|\,z)}\right]$$

$$= \mathbb{E}_{x,y\sim p; z\sim p_\theta}\left[\log \frac{p(y\,|\,x)\, p_\theta(z\,|\,x)}{q(z)\, q_\phi(y\,|\,z)}\right]$$

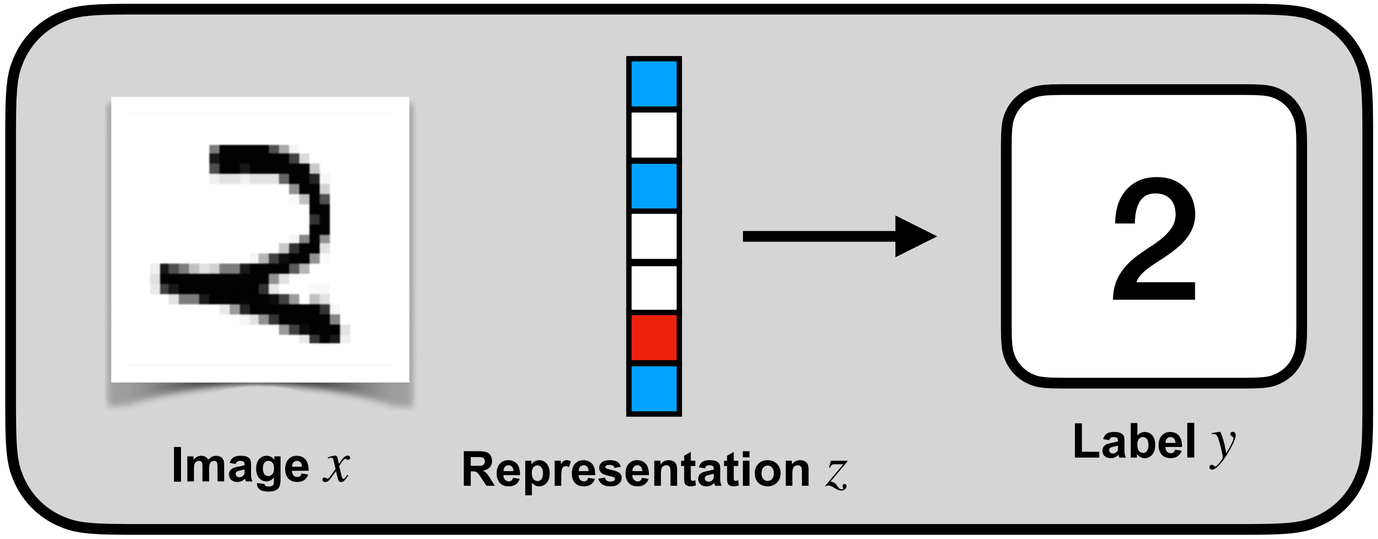
$$= \cancel{\mathbb{E}_{x,y\sim p; z\sim p_\theta}\big[\log p(y\,|\,x)\big]}$$
$$- \mathbb{E}_{x,y\sim p; z\sim p_\theta}\big[\log q_\phi(y\,|\,z)\big]$$
$$+ \mathbb{E}_{x,y\sim p; z\sim p_\theta}\left[\log \frac{p_\theta(z\,|\,x)}{q(z)}\right]$$

$$\mathscr{L} = - \mathbb{E}_{x,y\sim p; z\sim p_\theta}\big[\log q_\phi(y\,|\,z)\big] + D_{KL}\big[p_\theta(z\,|\,x) \,\|\, q(z)\big]$$

Real world P : $p(x,y,z) = p(x,y)\, p_\theta(z\,|\,x)$



Fantasy world: Q : $q(x,y,z) = p(x)\, q(z)\, q_\phi(y\,|\,z)$



Adjust seasonings, salt to taste

$$\mathcal{L} = -\mathbb{E}_{x,y\sim p;z\sim p_\theta}\left[\log q_\phi(y\,|\,z)\right] + D_{KL}\left[p_\theta(z\,|\,x)\,\|\,q(z)\right]$$

- KL divergence is a regularization on the learned representations (analogous to L2 regularization via a Gaussian prior on model weights)
- Typical to use a weight $\beta \leq 1$ on the KL Divergence term to balance trade off between prediction and regularization
- Closely related to *Information Bottleneck* objective $\max I(z, y; \theta) - \beta\, I(z, x\,|\,\theta)$
- **Gaussian Prior:** $D_{KL}\left[\mathcal{N}(\mu, \sigma^2)\,\|\,\mathcal{N}(0,1)\right]$ on the board

More representations



That’s all!

- Slides will be posted on the course website: https://colen-teaching.github.io/DATA442-Fall-2026
- Lab demos will also be posted
- ***PS3*** due November 8
- **REMEMBER:** TA Office Hours Tuesday 12-1pm, Student Success Center